

Práctica 2 Portal web y carrito de compras Urban Style

<https://github.com/Gabi459/urban-style.git>

index.html

```
<!DOCTYPE html>
<html lang="es">

<head>

    <!-- Permitimos tildes, ñ, € y otros caracteres -->
    <meta charset="UTF-8">

    <!-- Hacemos que la página se adapte a móviles y tablets -->
    <meta name="viewport" content="width=device-width,
initial-scale=1">

    <!-- Título que aparece en la pestaña -->
    <title>Urban Style</title>

    <!-- Conectamos el archivo CSS -->
    <link rel="stylesheet" href="styles.css">

    <!-- Script de analítica externa (Google Analytics) -->
    <!-- async hace que se descargue sin bloquear la página y se
ejecute en cuanto llegue -->
    <script async
src="https://www.googletagmanager.com/gtag/js?id=G-XXXXXXXXXX"></script
>

    <!-- Configuramos Google Analytics -->
    <script>
        window.dataLayer = window.dataLayer || [];

        function gtag() {
            dataLayer.push(arguments);
        }

        gtag('js', new Date());
        gtag('config', 'G-XXXXXXXXXX');
    </script>

    <!-- Conectamos el archivo JavaScript -->
```

```
<!-- defer hace que se ejecute cuando el HTML ya está cargado, así  
el DOM está disponible -->  
<script src="app.js" defer></script>  
  
</head>  
  
<body>  
  
  <!-- CABECERA DE SESIÓN -->  
  <header class="cabecera">  
  
    <!-- Nombre de la tienda -->  
    <h1>Urban Style</h1>  
  
    <!-- Información de la sesión -->  
    <div class="info-cabecera">  
  
      <!-- Fecha actual -->  
      <span id="fecha"></span>  
  
      <!-- Saludo del usuario -->  
      <span id="saludo"></span>  
  
      <!-- Tipo de usuario -->  
      <span id="rol" class="etiqueta-rol"></span>  
  
      <!-- Estado de la conexión -->  
      <span id="conexion" class="conexion"></span>  
  
    </div>  
  
  </header>  
  
  <!-- CONTENIDO PRINCIPAL -->  
  <main>  
  
    <!-- PERFIL DEL USUARIO Y SESIÓN -->  
    <section class="tarjeta">  
  
      <h2>Perfil y sesión</h2>
```

```
<!-- Datos principales del usuario -->
<div class="datos-perfil">

    <!-- Email -->
    <div class="dato">
        <span class="etiqueta">Email</span>
        <span id="email"></span>
    </div>

    <!-- Nombre de usuario del email -->
    <div class="dato">
        <span class="etiqueta">Usuario</span>
        <span id="usuario"></span>
    </div>

    <!-- Dominio del email -->
    <div class="dato">
        <span class="etiqueta">Dominio</span>
        <span id="dominio"></span>
    </div>

    <!-- ID del cliente -->
    <div class="dato">
        <span class="etiqueta">ID Cliente</span>
        <span id="idCliente"></span>
    </div>

    <!-- Apodo -->
    <div class="dato">
        <span class="etiqueta">Apodo</span>
        <span id="apodo"></span>
    </div>

    <!-- Tipo de membresía -->
    <div class="dato">
        <span class="etiqueta">Membresía</span>
        <span id="membresia"></span>
    </div>

    <!-- Número de prendas de regalo -->
    <div class="dato">
        <span class="etiqueta">Prendas de regalo</span>
        <span id="prendasRegalo"></span>
    </div>
</div>
```

```

        </div>

        <!-- Idioma del navegador -->
        <div class="dato">
            <span class="etiqueta">Idioma</span>
            <span id="idioma"></span>
        </div>

        <!-- ID de la sesión -->
        <div class="dato">
            <span class="etiqueta">ID Sesión</span>
            <span id="sesionId"></span>
        </div>

    </div>

</section>

<!-- BANNER DE OFERTA RELÁMPAGO -->
<section id="oferta" class="tarjeta oferta">

    <h2>Oferta Relámpago</h2>

    <p>Descuento exclusivo del 20% durante 15 segundos.</p>

    <!-- Contador de segundos -->
    <strong id="contadorOferta" class="contador">15</strong>

    <!-- Mensaje del estado de la oferta -->
    <p id="estadoOferta">Oferta disponible</p>

    <!-- Botón para activar la oferta -->
    <button id="activarOferta" type="button" class="boton">
        Activar Oferta Relámpago
    </button>

</section>

<!-- TIENDA -->
<section class="tienda">

```

```
<!-- CATÁLOGO -->
<div class="tarjeta">

    <h2>Catálogo</h2>

    <!-- Aquí JavaScript añada las prendas -->
    <div id="catalogoProductos" class="productos"></div>

</div>

<!-- CARRITO -->
<div class="tarjeta">

    <h2>Carrito</h2>

    <!-- Cupón de descuento -->
    <label for="cupon">Cupón de descuento (€)</label>

    <div class="cupon">
        <input type="text" id="cupon" placeholder="Ejemplo:
10">

        <button id="aplicarCupon" type="button"
class="boton">Aplicar</button>
    </div>

    <!-- DESGLOSE DE LA COMPRA -->
    <div class="desglose">

        <!-- Precio antes de descuentos -->
        <div class="linea-precio">
            <span>Subtotal</span>
            <span id="subtotal">0,00 €</span>
        </div>

        <!-- Descuento del cupón -->
        <div class="linea-precio">
            <span>Descuento</span>
            <span id="descuento">0,00 €</span>
        </div>

    </div>

</div>
```

```

        <!-- IVA -->
        <div class="linea-precio">
            <span>IVA (21%)</span>
            <span id="iva">0,00 €</span>
        </div>

        <!-- Precio final -->
        <div class="linea-precio total">
            <span>Total a pagar</span>
            <strong id="total">0,00 €</strong>
        </div>

    </div>

    <!-- Número del pedido actual -->
    <p class="pedido">
        Número de pedido: <strong
id="numeroPedido">1</strong>
    </p>

    <!-- Botón para procesar el pedido -->
    <button id="procesarPedido" type="button"
class="boton">
        Procesar pedido
    </button>

    <!-- Mensaje de aviso del carrito -->
    <p id="avisoCarrito" class="aviso" role="alert"></p>

</div>

</section>

<!-- SECCIÓN DE OPINIONES -->
<section class="tarjeta">

    <h2>Opiniones</h2>

    <!-- FORMULARIO PARA ESCRIBIR UNA RESEÑA -->
    <form id="formularioResena">

```

```

        <label for="textoResena">Tu opinión</label>

        <textarea id="textoResena" rows="4"
placeholder="Escribe tu opinión sobre nuestras prendas..."></textarea>

        <!-- Botón para enviar la reseña -->
        <button type="submit" class="boton">Publicar
opinión</button>

        <!-- Mensaje de aviso -->
        <p id="avisoResena" class="aviso" role="alert"></p>

    </form>

    <!-- JavaScript coloca aquí las reseñas -->
    <div id="listaResenas" class="lista-resenas"></div>

</section>

</main>

<!-- PIE DE PÁGINA -->
<footer class="pie">
    <p>© 2026 Urban Style. Todos los derechos reservados.</p>
</footer>

</body>

</html>

```

[app.js](#)

```

'use strict';

/* =====
Urban Style · Portal web y carrito de compras (MVP)
app.js: lógica principal del portal.

Se carga con el atributo "defer" en el <head>, así que cuando
este código se ejecuta el DOM ya está completamente disponible.

```

```

===== */

/* CONFIGURACIÓN Y DATOS, creo unas constantes para guardar los datos
que se utilizan durante el programa */

const IVA = 0.21; // iva del 21%
const DURACION_OFERTA = 15; // duración de la oferta en segundos
const CLAVE_RESENAS = 'urbanStyle.resenas'; // clave para guardar las
reseñas en localStorage

// Datos de sesión simulados: pongo unos datos de ejemplo de un
usuario.
// Algunos los pongo mal o vacíos para poder limpiarlos (1.3) y poner
valores por defecto (1.4)
const datosSesion = {
  email: ' Gabriel.Martin@UrbanStyle.ES ', // con espacios y
mayúsculas para limpiarlo
  idCliente: 1234, // se tiene que ver como 001234
  apodo: '', // vacío -> "Cliente VIP"
  membresia: undefined, // no definida -> "Básica"
  prendasRegalo: 0, // tiene 0 de verdad, así que NO se cambia a 2
};

// Catálogo de la tienda. Los precios vienen en texto, como pide el
enunciado.
const catalogo = [
  { nombre: 'Chaqueta Denim', precio: '59.90€' },
  { nombre: 'Camiseta Urban', precio: '19.99€' },
  { nombre: 'Sudadera Oversize', precio: '44.50€' },
  { nombre: 'Gorra Street', precio: '15.00€' },
];

// Variables que van cambiando mientras se usa la página
let usuarioActual = 'Invitado';
let pedidoActual = 1; // número del pedido actual
let ofertaActiva = false; // indica si hay una cuenta atrás en marcha
let intervaloOferta = null; // guarda el setInterval para poder pararlo
let resenas = []; // lista de reseñas

// Creo una función para buscar elementos del HTML usando solo su id
const $ = (id) => document.getElementById(id);

```



```

// Función para mostrar un mensaje de aviso. El tipo ('ok' o 'error')
cambia el color con CSS
function mostrarAviso(id, texto, tipo) {
    $(id).textContent = texto;
    $(id).dataset.tipo = tipo;
}

/* BLOQUE 1 Inicio de sesión y diagnóstico del entorno */

// 1.2 Recojo el usuario y el rol de los parámetros de la URL.
// Si no vienen en la URL, get() devuelve null y con ?? pongo los
valores por defecto.
function leerParametrosURL() {
    const parametros = new URLSearchParams(window.location.search);

    const usuario = parametros.get('usuario') ?? 'Invitado';
    const rol = parametros.get('rol') ?? 'invitado';

    return { usuario, rol };
}

// 1.2 Función que obtiene información del navegador: idioma, conexión,
un id de sesión y la fecha.
function diagnosticarEntorno() {
    return {
        idioma: navigator.language, // idioma del navegador, por ejemplo
"es-ES"
        online: navigator.onLine, // true si hay conexión a internet
        idSesion: crypto.randomUUID(), // identificador único y seguro
        // Fecha en texto largo en español, por ejemplo "jueves, 1 de
octubre de 2026"
        fecha: new Date().toLocaleDateString('es-ES', {
            weekday: 'long',
            day: 'numeric',
            month: 'long',
            year: 'numeric',
        }),
    };
}

// 1.3 Limpia el correo quitando los espacios y pasándolo a minúsculas.

```

```

// Después separo la parte del usuario y la del dominio usando la
posición de la @.
function sanitizarEmail(emailOriginal) {
  const email = emailOriginal.trim().toLowerCase();
  const posicionArroba = email.indexOf('@');

  return {
    email,
    usuario: email.slice(0, posicionArroba),
    dominio: email.slice(posicionArroba + 1),
  };
}

// 1.3 Convierte el id del cliente en un texto de 6 dígitos añadiendo
ceros delante.
function formatearIdCliente(id) {
  return String(id).padStart(6, '0');
}

// 1.4 Pongo los valores por defecto con el operador que corresponde en
cada caso.
function aplicarPreferencias(perfil) {
  // ||= asigna si el valor es falso, y una cadena vacía lo es ->
  "Cliente VIP"
  perfil.apodo ||= 'Cliente VIP';

  // ??= solo asigna si es null o undefined -> "Básica"
  perfil.membresia ??= 'Básica';

  // También uso ??= porque con ||= un 0 se cambiaría a 2, y el 0 es un
saldo real
  perfil.prendasRegalo ??= 2;

  return perfil;
}

// Función que inicia toda la información del usuario y la muestra en
la página
function iniciarSesion() {
  const { usuario, rol } = leerParametrosURL();
  const entorno = diagnosticarEntorno();
  const correo = sanitizarEmail(datosSesion.email);
  const perfil = aplicarPreferencias(datosSesion);

```

```

    usuarioActual = usuario; // lo guardo para usarlo en las reseñas

    // Cabecera
    $('fecha').textContent = entorno.fecha;
    $('saludo').textContent = `Hola, ${usuario}`;
    $('rol').textContent = rol;
    $('conexion').textContent = entorno.online ? 'En línea' : 'Sin
conexión';
    $('conexion').dataset.estado = entorno.online ? 'online' : 'offline';

    // Perfil
    $('email').textContent = correo.email;
    $('usuario').textContent = correo.usuario;
    $('dominio').textContent = correo.dominio;
    $('idCliente').textContent = formatearIdCliente(perfil.idCliente);
    $('apodo').textContent = perfil.apodo;
    $('membresia').textContent = perfil.membresia;
    $('prendasRegalo').textContent = perfil.prendasRegalo;
    $('idioma').textContent = entorno.idioma;
    $('sesionId').textContent = entorno.idSesion;
}

/* BLOQUE 2 Catálogo, operaciones financieras y formato de moneda */

// 2.4 Formato de moneda en euros para España (ejemplo: 84,57 €)
const formatoEuros = new Intl.NumberFormat('es-ES', { style:
'currency', currency: 'EUR' });

// 2.1 parseFloat lee el número del principio del texto y se para en el
€. "59.90€" -> 59.9
function extraerPrecio(textoPrecio) {
    return parseFloat(textoPrecio);
}

// 2.1 Suma los precios de las prendas que están marcadas en el
catálogo
function calcularSubtotal() {
    const marcadas = document.querySelectorAll('#catalogoProductos
input:checked');

    return Array.from(marcadas).reduce(

```

```

    (suma, casilla) => suma + extraerPrecio(casilla.value),
    0
  );
}

// 2.2 y 2.4 Calcula el desglose de la compra y lo muestra en euros.
// Primero compruebo que el subtotal es un número válido, después
// convierto el cupón
// a número, lo resto y calculo el iva sobre lo que queda.
function actualizarCarrito() {
  const subtotal = calcularSubtotal();

  // Number.isFinite descarta NaN e Infinity, así no se opera con datos
  // incorrectos
  if (!Number.isFinite(subtotal)) {
    mostrarAviso('avisoCarrito', 'El subtotal no es un número válido.',
    'error');
    return false;
  }

  // El cupón llega como texto ("10"), con Number() lo paso a número.
  // Si está vacío vale 0
  const cupon = Number($('cupon').value.trim());

  if (!Number.isFinite(cupon) || cupon < 0) {
    mostrarAviso('avisoCarrito', 'El cupón tiene que ser un número, por
    ejemplo 10.', 'error');
    return false;
  }

  // El descuento no puede ser mayor que la compra, así la base nunca
  // es negativa
  const descuento = Math.min(cupon, subtotal);
  const baseImponible = subtotal - descuento; // resto el cupón
  const iva = baseImponible * IVA; // 21% sobre lo que queda después
  // del descuento
  const total = baseImponible + iva;

  // Muestro el desglose con el formato de euros
  $('subtotal').textContent = formatoEuros.format(subtotal);
  $('descuento').textContent = formatoEuros.format(descuento);
  $('iva').textContent = formatoEuros.format(iva);
  $('total').textContent = formatoEuros.format(total);
}

```

```

    mostrarAviso('avisoCarrito', '', '');
    return true;
}

// 2.3 Al procesar el pedido aumento en uno el número del pedido
function procesarPedido() {
    if (calcularSubtotal() === 0) {
        mostrarAviso('avisoCarrito', 'Selecciona al menos una prenda.',
        'error');
        return;
    }

    // Si el desglose tiene algún error no se procesa el pedido
    if (!actualizarCarrito()) return;

    mostrarAviso('avisoCarrito', `Pedido nº ${pedidoActual} procesado
    correctamente.`, 'ok');

    pedidoActual += 1;
    $('#numeroPedido').textContent = pedidoActual;
}

// Creo las prendas del catálogo con una casilla cada una.
// Cuando el usuario marca o desmarca una prenda se actualiza el
carrito.
function iniciarCarrito() {
    catalogo.forEach((prenda) => {
        const etiqueta = document.createElement('label');
        etiqueta.className = 'producto';

        const casilla = document.createElement('input');
        casilla.type = 'checkbox';
        casilla.value = prenda.precio; // guardo el precio en texto, tal
cual viene
        casilla.addEventListener('change', actualizarCarrito);

        const nombre = document.createElement('span');
        nombre.textContent = prenda.nombre;

        const precio = document.createElement('strong');
        precio.textContent = prenda.precio;
    });
}

```

```

        etiqueta.append(casilla, nombre, precio);
        $('catalogoProductos').append(etiqueta);
    });

    $('aplicarCupon').addEventListener('click', actualizarCarrito);
    $('procesarPedido').addEventListener('click', procesarPedido);
}

/* BLOQUE 3 Oferta relámpago (temporizador con control) */

// 3.1 y 3.2 Función que activa la cuenta atrás. Cada segundo resta uno
al contador.
function activarOferta() {
    // Si ya hay una oferta en marcha no hago nada.
    // Así no se crean temporizadores duplicados ni se aceleran los
segundos.
    if (ofertaActiva) return;

    ofertaActiva = true;
    let segundos = DURACION_OFERTA;

    $('contadorOferta').textContent = segundos;
    $('estadoOferta').textContent = ';Oferta activa!';
    $('oferta').dataset.estado = 'activa';

    intervaloOferta = setInterval(() => {
        segundos -= 1;
        $('contadorOferta').textContent = segundos;

        if (segundos === 0) {
            finalizarOferta();
        }
    }, 1000);
}

// 3.3 Cuando llega a 0 paro el temporizador, vuelvo a poner
ofertaActiva en false
// para que se pueda activar otra vez y aviso de que la oferta ha
terminado.
function finalizarOferta() {
    clearInterval(intervaloOferta);
    intervaloOferta = null;
}

```

```

ofertaActiva = false;

$('estadoOferta').textContent = 'La oferta ha expirado.';
$('oferta').dataset.estado = 'expirada';
}

/* BLOQUE 4 Reseñas, seguridad y persistencia */

// 4.3 Recupera las reseñas guardadas en localStorage.
// Uso try/catch porque si los datos guardados están mal, JSON.parse da
error y se rompería la página.
function cargarResenas() {
  try {
    const guardadas = localStorage.getItem(CLAVE_RESENAS);
    if (guardadas === null) return []; // todavía no hay nada guardado

    const datos = JSON.parse(guardadas);
    return Array.isArray(datos) ? datos : []; // compruebo que lo
guardado es una lista
  } catch (error) {
    console.error('Error al cargar las reseñas:', error);
    mostrarAviso('avisoResena', 'No se pudieron cargar las reseñas
guardadas.', 'error');
    return [];
  }
}

// 4.3 Guarda la lista de reseñas en localStorage para que no se
pierdan al recargar.
// También uso try/catch porque guardar puede fallar (por ejemplo si el
almacenamiento está lleno).
// Devuelve true si se ha guardado y false si no.
function guardarResenas() {
  try {
    localStorage.setItem(CLAVE_RESENAS, JSON.stringify(resenas));
    return true;
  } catch (error) {
    console.error('Error al guardar las reseñas:', error);
    return false;
  }
}

```

```

// 4.2 Creo el objeto de la reseña con el id (marca de tiempo), el
usuario, la hora y el comentario
function crearResena(comentario) {
  return {
    id: Date.now(),
    usuario: usuarioActual,
    hora: new Date().toLocaleTimeString('es-ES'),
    comentario,
  };
}

// 4.4 Muestra las reseñas creando los elementos desde JavaScript.
// Uso textContent y no innerHTML, así si alguien escribe <script> se
ve como texto y no se ejecuta.
function mostrarResenas() {
  const lista = $('listaResenas');
  lista.replaceChildren(); // vacío la lista antes de volver a pintarla

  resenas.forEach((resena) => {
    const tarjeta = document.createElement('article');
    tarjeta.className = 'resena';

    const autor = document.createElement('strong');
    autor.textContent = `${resena.usuario} · ${resena.hora}`;

    const texto = document.createElement('p');
    texto.textContent = resena.comentario;

    tarjeta.append(autor, texto);
    lista.append(tarjeta);
  });
}

// 4.1 Cuando se envía el formulario creo la reseña, la guardo y
actualizo la lista
function publicarResena(evento) {
  evento.preventDefault(); // evita que la página se recargue al enviar
el formulario

  const comentario = $('textoResena').value.trim();

  if (comentario === '') {

```



```

        mostrarAviso('avisoResena', 'Escribe tu opinión antes de
publicarla.', 'error');
        return;
    }

    resenas.push(crearResena(comentario));
    const guardada = guardarResenas();
    mostrarResenas();

    $('textoResena').value = '';

    if (guardada) {
        mostrarAviso('avisoResena', 'Opinión publicada.', 'ok');
    } else {
        mostrarAviso('avisoResena', 'Opinión publicada, pero no se pudo
guardar.', 'error');
    }
}

/* ARRANQUE */
// Al cargar la página inicio todas las partes: la sesión, el carrito,
la oferta y las reseñas.
iniciarSesion();
iniciarCarrito();
$('activarOferta').addEventListener('click', activarOferta);
resenas = cargarResenas();
mostrarResenas();
$('formularioResena').addEventListener('submit', publicarResena);

```

styles.css

```

/* =====
Urban Style · Hoja de estilos
Estética urbana: hormigón, denim y amarillo de señalización.
===== */

/* Colores y valores que se repiten en toda la página */
:root {
    --hormigon: #E2E3DE;
    --superficie: #FBFBF9;
    --tinta: #1C1F24;

```

```

--tinta-suave: #585E68;
--linea: #C9CBC4;
--denim: #2B4A6F;
--denim-oscuro: #1E3550;
--senal: #F4C21B;
--ok: #2F7D4F;
--error: #B42318;

--sombra: 4px 4px 0 var(--tinta);
--radio: 6px;
}

* {
  box-sizing: border-box;
}

/* Tipografía clara y fondo general */
body {
  margin: 0;
  font-family: "Helvetica Neue", Arial, sans-serif;
  line-height: 1.5;
  color: var(--tinta);
  background: var(--hormigon);
}

h1, h2 {
  margin: 0 0 0.8rem;
  font-weight: 800;
  line-height: 1.1;
}

/* ----- Cabecera de sesión ----- */
.cabecera {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 1rem 1.5rem;
  background: var(--tinta);
  color: #fff;
}

```

```
.cabecera h1 {
  margin: 0;
}

.info-cabecera {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.8rem;
  font-size: 0.9rem;
}

/* Rol del usuario en forma de etiqueta redonda */
.etiqueta-rol {
  padding: 0.1rem 0.6rem;
  border: 1px solid #fff;
  border-radius: 999px;
}

/* Indicador de conexión: un punto verde o rojo según el estado */
.conexion::before {
  content: "";
  display: inline-block;
  width: 0.6rem;
  height: 0.6rem;
  margin-right: 0.4rem;
  border-radius: 50%;
  background: var(--linea);
}

.conexion[data-estado="online"]::before {
  background: #4CC37A;
}

.conexion[data-estado="offline"]::before {
  background: #F0645A;
}

/* ----- Contenido principal ----- */
main {
  display: grid;
  gap: 1.5rem;
  max-width: 1100px;
}
```

```
margin: 0 auto;
padding: 1.5rem;
}

/* Diseño en tarjetas */
.tarjeta {
padding: 1.5rem;
background: var(--superficie);
border: 2px solid var(--tinta);
border-radius: var(--radio);
box-shadow: var(--sombra);
}

/* ----- Perfil ----- */
.datos-perfil {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
gap: 1rem;
}

.dato {
display: flex;
flex-direction: column;
font-weight: 600;
overflow-wrap: anywhere;
}

.etiqueta {
color: var(--tinta-suave);
font-size: 0.8rem;
font-weight: 400;
}

/* ----- Oferta relámpago ----- */
.oferta {
background: var(--senal);
}

.contador {
display: block;
margin: 0.5rem 0;
font-size: 4rem;
line-height: 1;
```

```

}

/* Cuando la oferta termina el banner se pone gris */
.oferta[data-estado="expirada"] {
  background: #D6D7D2;
}

/* ----- Tienda: catálogo + carrito ----- */
.tienda {
  display: grid;
  gap: 1.5rem;
}

/* En pantallas grandes el catálogo y el carrito se ponen uno al lado
del otro */
@media (min-width: 800px) {
  .tienda {
    grid-template-columns: 1fr 1fr;
    align-items: start;
  }
}

.productos {
  display: grid;
  gap: 0.6rem;
}

/* Cada prenda del catálogo */
.producto {
  display: grid;
  grid-template-columns: auto 1fr auto;
  align-items: center;
  gap: 0.8rem;
  padding: 0.8rem 1rem;
  border: 2px solid var(--linea);
  border-radius: var(--radio);
  cursor: pointer;
}

/* Prenda marcada */
.producto:has(input:checked) {
  border-color: var(--denim);
  background: #EEF2F7;
}

```

```

}

/* Campos de texto */
label {
    display: block;
    margin-bottom: 0.3rem;
    font-weight: 600;
}

input[type="text"],
textarea {
    width: 100%;
    padding: 0.6rem 0.8rem;
    font: inherit;
    border: 2px solid var(--tinta);
    border-radius: 4px;
}

.cupon {
    display: flex;
    gap: 0.5rem;
}

/* Desglose de la compra */
.desglose {
    margin: 1.2rem 0;
}

.linea-precio {
    display: flex;
    justify-content: space-between;
    padding: 0.5rem 0;
    border-bottom: 1px dashed var(--linea);
}

.linea-precio.total {
    border-bottom: none;
    border-top: 2px solid var(--tinta);
    font-size: 1.3rem;
    font-weight: 800;
}

/* ----- Botones de acción ----- */

```

```
.boton {
  padding: 0.7rem 1.2rem;
  font: inherit;
  font-weight: 700;
  color: #fff;
  background: var(--denim);
  border: 2px solid var(--tinta);
  border-radius: 4px;
  box-shadow: var(--sombra);
  cursor: pointer;
}

.boton:hover {
  background: var(--denim-oscuro);
}

/* Efecto de pulsar el botón */
.boton:active {
  transform: translate(2px, 2px);
  box-shadow: 2px 2px 0 var(--tinta);
}

/* ----- Avisos (indicadores de estado) ----- */
.avisos:empty {
  display: none; /* si no hay mensaje no se ve */
}

.avisos {
  margin-top: 1rem;
  padding: 0.6rem 0.9rem;
  border-left: 4px solid var(--denim);
  background: #EEF2F7;
}

.avisos[data-tipo="ok"] {
  border-color: var(--ok);
  background: #E9F4EE;
}

.avisos[data-tipo="error"] {
  border-color: var(--error);
  background: #FBECEA;
}
```

```

/* ----- Opiniones ----- */
#formularioResena textarea {
  margin-bottom: 0.8rem;
}

.lista-resenas {
  display: grid;
  gap: 0.8rem;
  margin-top: 1.5rem;
}

.resena {
  padding: 0.8rem 1rem;
  border-left: 4px solid var(--denim);
  background: #F1F2EE;
}

.resena p {
  margin: 0.3rem 0 0;
  white-space: pre-wrap; /* respeta los saltos de línea del comentario */
  overflow-wrap: anywhere;
}

/* ----- Pie ----- */
.pie {
  padding: 1.5rem;
  text-align: center;
  color: var(--tinta-suave);
  font-size: 0.85rem;
}

```

[README.md](#)

Urban Style: portal web y carrito de compras (MVP)

Aplicación web cliente para la tienda de moda **Urban Style**, hecha con HTML5 semántico, CSS y JavaScript (`'use strict`).

Estructura


```
...
```

```
urban-style/
```

```
|— index.html   Estructura de la página y carga de scripts (async /  
defer)
```

```
|— styles.css   Diseño en tarjetas, botones, tipografía e indicadores  
de estado
```

```
|— app.js       Lógica de los 4 bloques
```

```
|— README.md
```

```
...
```

Cómo probarlo

Abre `index.html` en el navegador. Para simular un inicio de sesión, añada parámetros a la URL:

```
...
```

```
index.html?usuario=Ana&rol=admin
```

```
...
```

Sin parámetros, el portal usa los valores por defecto `Invitado` / `invitado`.

Dónde se resuelve cada requisito

```
| Requisito | Solución |
```

```
|---|---|
```

```
| 1.1 Carga de scripts | Analítica con async y lógica con defer,  
los dos en el <head> |
```

```
| 1.2 Usuario y rol | URLSearchParams con valores por defecto |
```

```
| 1.2 Idioma y conexión | navigator.language y navigator.onLine |
```

```
| 1.2 ID de sesión | crypto.randomUUID() |
```

```
| 1.2 Fecha en español | toLocaleDateString('es-ES', {...}) |
```

```
| 1.3 Limpieza del correo | trim() y toLowerCase(), separación con  
indexOf y slice |
```

```
| 1.3 ID de 6 dígitos | padStart(6, '0') |
```

```
| 1.4 Valores por defecto | \\|\\|= para el apodo, ??= para membresía  
y prendas de regalo (respeto el 0) |
```

```
| 2.1 Precios en texto | parseFloat("59.90€") y suma con reduce |
```

```
| 2.2 Validación e IVA | Number.isFinite, cupón con Number(), IVA  
del 21 % sobre la base |
```

```
| 2.3 Número de pedido | pedidoActual += 1 |
```

```
| 2.4 Formato en euros | `Intl.NumberFormat('es-ES', { style:
'currency', currency: 'EUR' })` |
| 3.1 Cuenta atrás | `setInterval` de 1000 ms desde 15 |
| 3.2 Clics repetidos | Variable `ofertaActiva` que bloquea nuevos
temporizadores |
| 3.3 Final de la oferta | `clearInterval`, `ofertaActiva = false` y
mensaje de expirada |
| 4.1 Formulario | Evento `submit` con `preventDefault()` |
| 4.2 Registro | `{ id: Date.now(), usuario, hora, comentario }` |
| 4.3 Persistencia | `localStorage` y `JSON` dentro de `try/catch` |
| 4.4 Anti-XSS | `createElement` y `textContent`, nunca `innerHTML` |
```

Cálculo de ejemplo

```
| Concepto | Importe |
|---|---|
| Chaqueta Denim + Camiseta Urban | 79,89 € |
| Cupón | 10,00 € |
| IVA 21 % (sobre 69,89 €) | 14,68 € |
| **Total a pagar** | **84,57 €** |
```

Autor

Gabriel Martín, práctica 2 de Prometeo by thePower.